

ELTE EÖTVÖS LORÁND UNIVERSITY

WEB DEVELOPMENT WITH MVC ARCHITECTURE

Modern web application development in .NET

Lecture 1

Dr. Máté Cserép

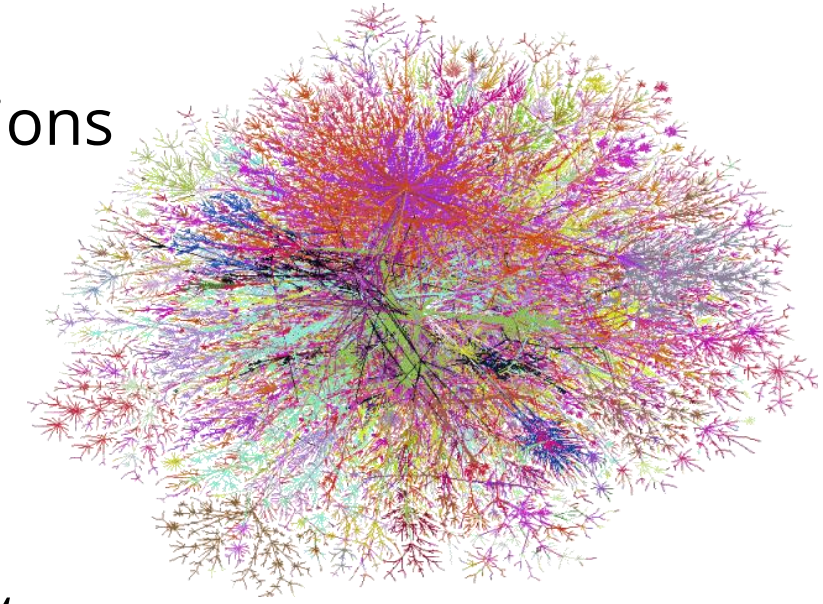
mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

NETWORK COMMUNICATION

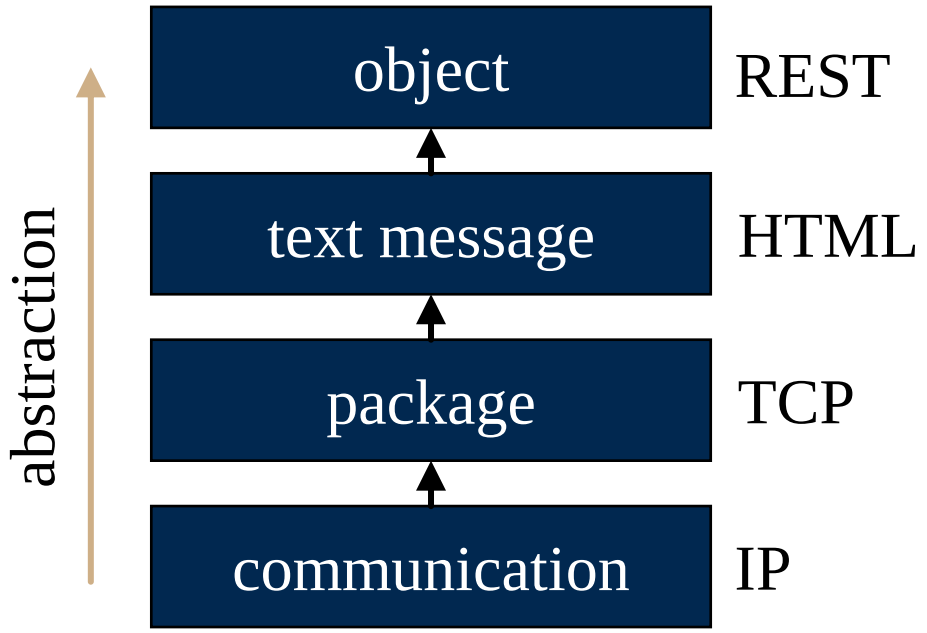
Applications often communicate over a network, usually based on the *OSI model*

- Which provides different communication solutions in a layered architecture
  - *application*: TLS, SSH, SMTP, etc.
  - *presentation*: HTML, CSS, JSON, etc.
  - *transport*: UDP, TCP
  - *network*: IP
- Additional communication and service *architectures* (MVC, REST) and *frameworks* (ASP.NET, WCF) are built on top of the model



NETWORK COMMUNICATION

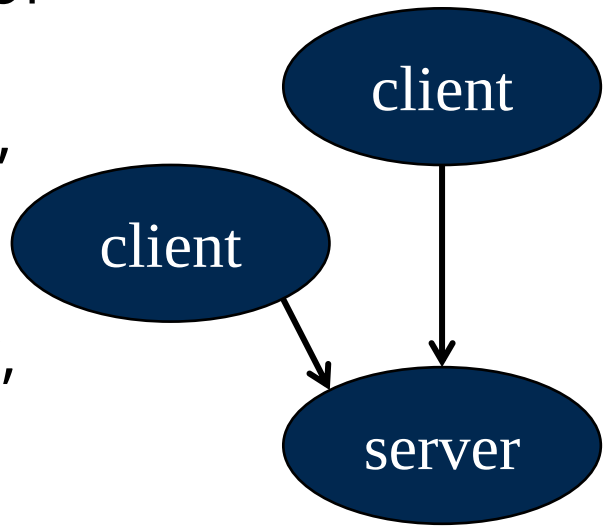
- Upper layers provide abstraction, masking the lower layers
  - Increases *security* and *reliability*
  - Ensures communication with complex content



NETWORK COMMUNICATION

Application systems that communicate over a network can be divided into two main categories:

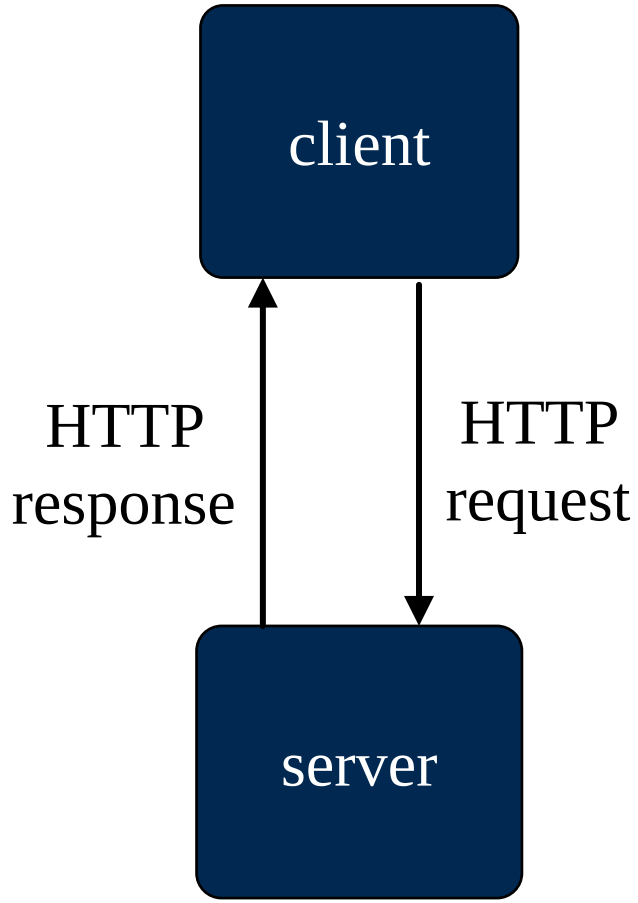
- *decentralized*, directly connected (P2P)
- *client-server*: a central machine serves any number of connected machines that may be:
  - *thick clients*: the execution is done by the client, the server is mainly used for communication and data management
  - *thin clients*: the execution is done by the server, the client is used for interaction



HTTP PROTOCOL

HTTP (Hypertext Transfer Protocol) is the standard protocol for data transfer on the web

- based on the request/response paradigm, i.e. the client sends a request to which the server responds
  - the response is interpreted and displayed by the client
  - the first line of the response is the *status line*, which gives feedback on the execution of the request (e.g. 200 successful, 404 not found, 503 service not available)



HTTP PROTOCOL

- Request can be of several types (*methods*), e.g.:
  - *GET*: a request for a given resource (associated with a path)
  - *POST*: send content with data (e.g. form)
- The protocol is *stateless*, i.e. requests to the server are independent of each other (no state-preservation between two requests)
  - at the same time, the server can establish a *session* with the client (the identification data is saved on the client side)
- The protocol can be made secure using TLS encryption (HTTPS)

DEVELOPMENT IN ASP.NET

Microsoft ASP.NET is an evolution of *ASP (Active Server Pages)* with .NET programming support

- a (server-side) programming interface for creating dynamic web pages and services over HTTP

ASP.NET Core is an open source, cross-platform framework based on .NET Core, which offers two programming models:

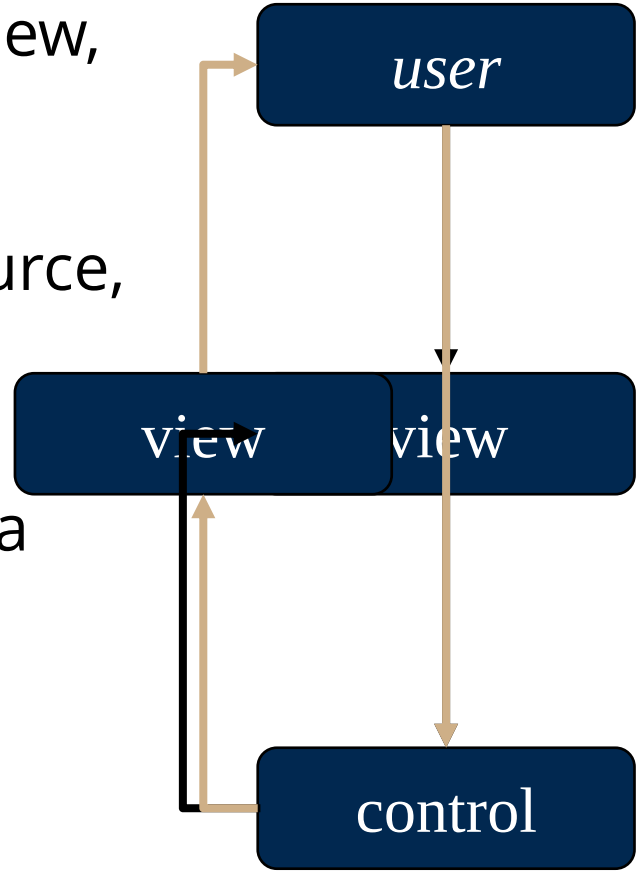
- *Razor Pages*: for simple, largely static applications that support dynamic functionality and data management, based on MVVM
- *ASP.NET MVC*: an architecture for developing complex, dynamic web pages, based on MVC

THE MVC ARCHITECTURE

In a *desktop environment*, the user connects to the view, which ensures the execution of the appropriate command (event handler, command)

In a *web environment*, the user connects to the resource, which is primarily determined by its path

- i.e. the user directly accesses the control
- the application must respond to the control with a (new) view associated with the resource

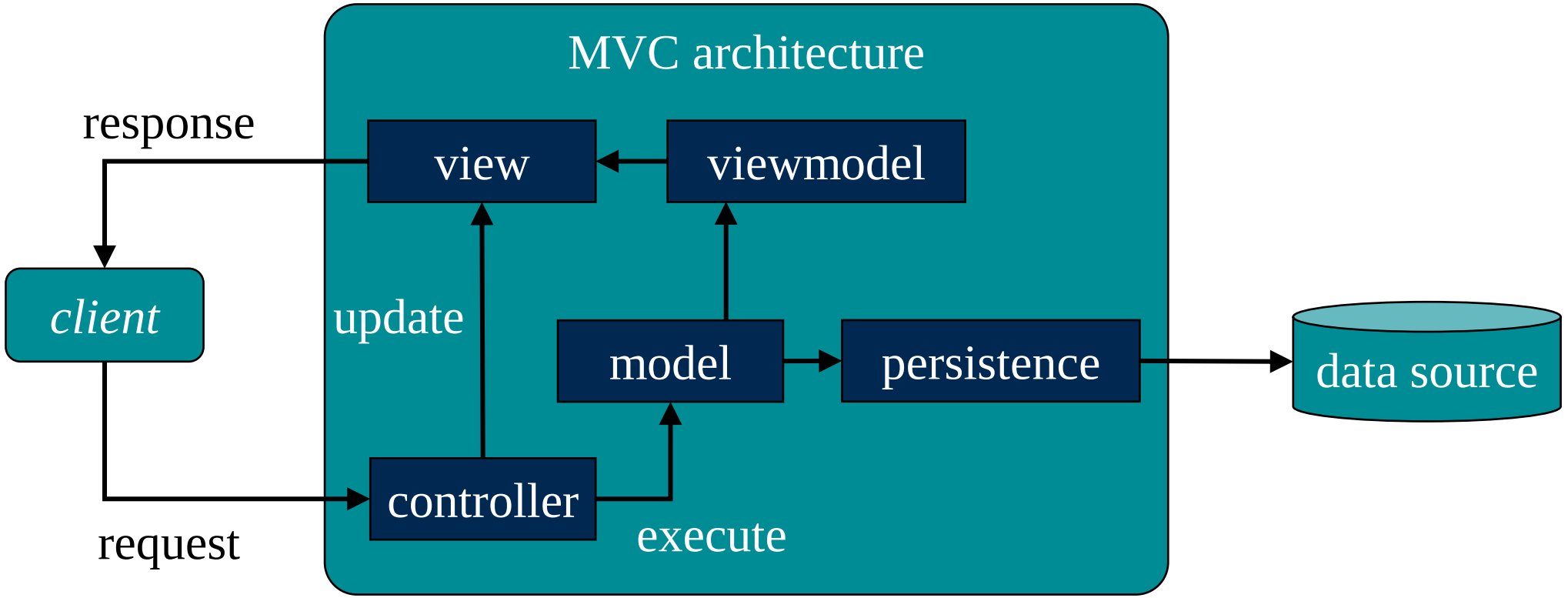


THE MVC ARCHITECTURE

The *Model/View/Controller (MVC)* architecture defines a multi-layered architecture that fits well with the web environment

- *controller* is the request server that provides the view based on the result of the request
- *view* is the mostly declarative definition of the user interface; it does not contain any business logic and presents the data
- *model* is the implementation of the business logic
- *viewmodel* is a gateway to represent data in a way that is appropriate for the view, so model and view representation of data is separated
- *persistence* is responsible for data access

THE MVC ARCHITECTURE

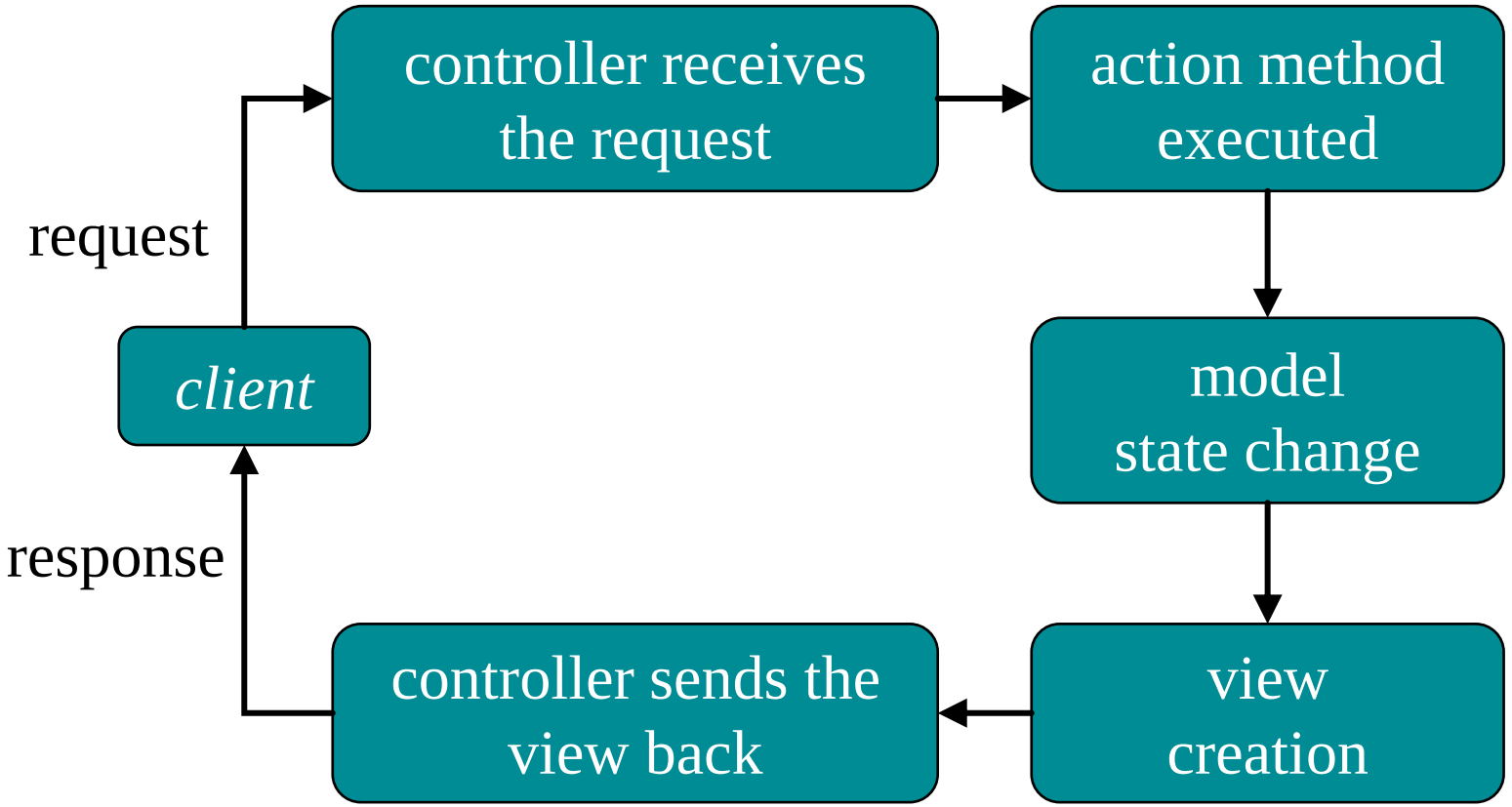


THE MVC ARCHITECTURE

Request process cycle in the MVC architecture:

1. the user makes a request to the server
2. the controller receives the request and executes the corresponding action in the model (*action method*)
3. the action executed in the model (may) causes a state change
4. the controller collects the *action result* and creates the new view (*push-based* approach)
    - another approach is to have the view retrieve the results from the controllers (*pull-based*)
    - data is transferred to the view using a view model
5. the user receives the response (view)

THE MVC ARCHITECTURE



COMPONENTS OF MVC APPLICATIONS

ASP.NET MVC applications implement the MVC architecture using dedicated components

- a view is a class (**`View`**) that is defined using a suitable markup language (e.g. *Razor*)
  - the view can use HTML and can reference the contents of the model (using data binding)
- a controller (subclass of **`Controller`**) is a class of activities in which actions (methods) are defined
  - the action result (**`IActionResult`**) is usually a view
- the model and persistence can be arbitrary

COMPONENTS OF MVC APPLICATIONS

Sample controller with multiple actions:

```
public class MyController : Controller {
  // controller
  public IActionResult Index() { // action
    return View("Index", "Welcome to the website!");
    // the result is a view with a simple string as the viewmodel
  }

  public IActionResult List(){
    ... // fetch some data from the model
    return View(list);
  }

  public IActionResult Details(String id){ … }
```

COMPONENTS OF MVC APPLICATIONS

Sample Razor view:

```
@model string @* the viewmodel is a simple string *@
@{ } @* code block *@

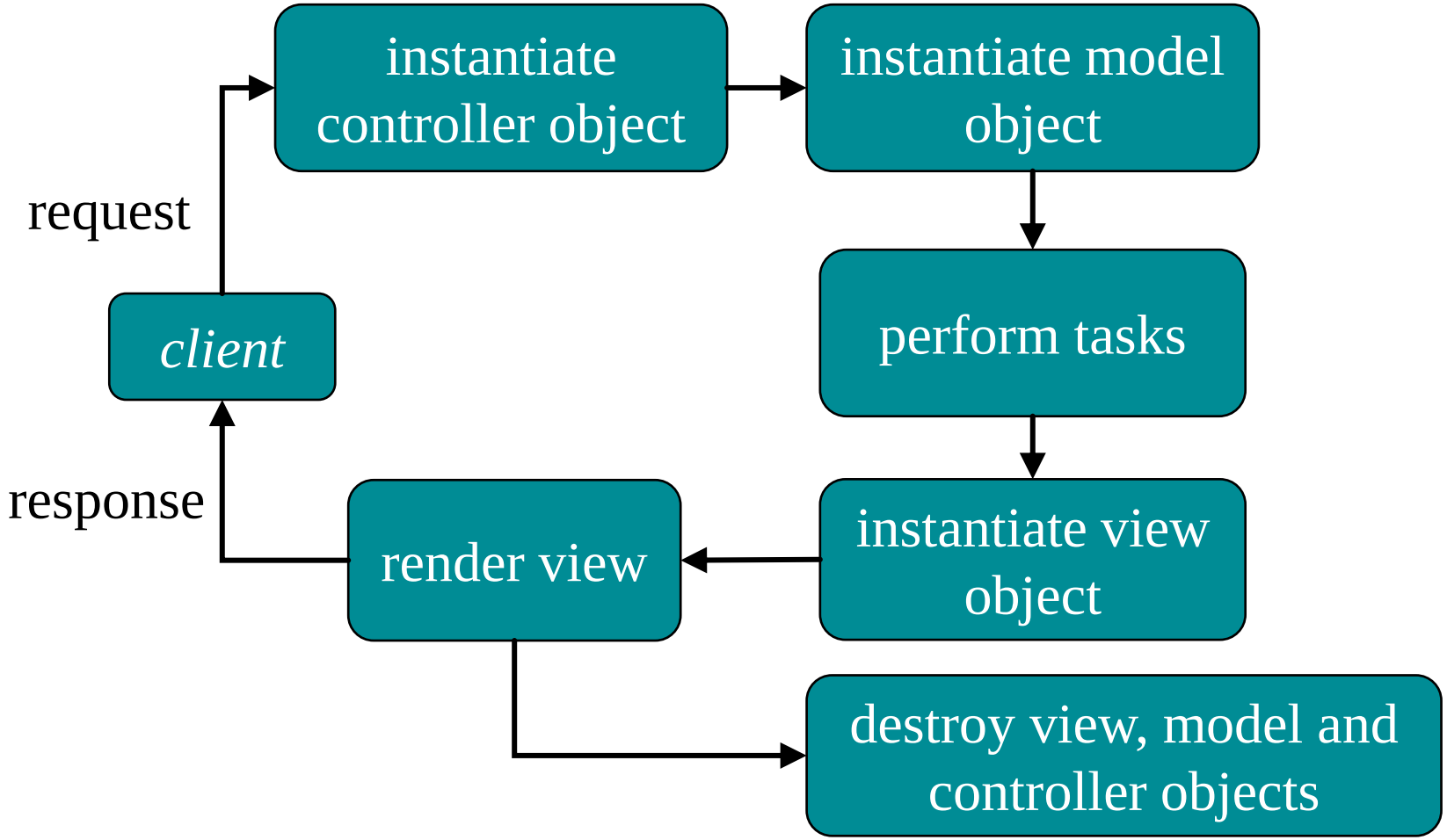
<!DOCTYPE html>
<html>
  <head>...</head> @* static content*@
  <body>
    <div>
      @Model @* display the viewmodel *@
    </div>
  </body>
</html>
```

LIFECYCLE OF WEB APPLICATIONS

The lifecycle of web applications is different from desktop and mobile applications

- the application can only respond to requests, requests are independent of each other and can arrive at any time
- *the application therefore does not keep state between two requests*
    - it is triggered by a request and instantiates objects
    - deletes objects when the request is served
    - certain data can remain in memory for a period of time
- *the persistence layer ensures that the data is preserved*

LIFECYCLE OF WEB APPLICATIONS



MVC PROJECT LAYOUT

The folder layout of the MVC application reflects the modular architecture

- the **`View`**, **`Controllers`** and **`Models`** folders contain the appropriate content
- the **`wwwroot`** directory holds the public static files (images, stylesheets, client-side scripts)
- the **`App_Data`** directory stores any data content (e.g. database files)
- the root contains the configuration (**`appsettings.json`**) and the application-level controls (**`Program.cs`**)

Additional software libraries can be installed using the NuGet package manager, as usual

ROUTING ENGINE

In an MVC architecture application, the user connects to the controller and runs its actions (with parameters)

- access and parameterization is provided by routes, which are controlled by a *routing engine*
- access is customizable (**`Program.cs`**), provided by default in the form **`<host>/<controller>/<action>/<parameters>`**
  - loads the default **`HomeController`** controller without specifying a controller
  - runs the **`Index`** action without specifying an action
  - parameters can be resolved in order or by name

ROUTING ENGINE

Examples:

Route	*Activity*
`http://myPage/`	`HomeController` → `Index`
`http://myPage/Hello`	`HelloController` → `Index`
`http://myPage/Hello/List`	`HelloController` → `List`
`http://myPage/Hello/Details/1` `http://myPage/Hello/Details?id=1`	`HelloController` → `Details` (with `id=1` parameter)

CONTROLLERS

Controllers are descendants of the Controller class, which implement actions using public operations

- the action returns a result (**`IActionResult`**), which can be
  - a view (**`ViewResult`**, **`PartialViewResult`**)
  - an error report (**`NotFoundResult`**, **`UnathorizedResult`**, **`StatusCodeResult`**)
  - redirect (**`RedirectResult`**)
  - file (**`FileResult`**), JSON (**`JsonResult`**), object (**`ObjectResult`**), other content (**`ContentResult`**)
  - empty (**`EmptyResult`**)

CONTROLLERS

E.g.:

```
public class MyController : Controller {
  ...
  public IActionResult LoadImage(String id){
    Byte[] image = … // load an image from the disk based on the Id
    if (image == null) // when no image with that Id was found
      return RedirectToAction("Error");
      // redirect to another action
    return File(image, "image/png");
    // return image as file content
}
public IActionResult Error() {
  return NotFound("Content not found.");
} // return error code
```

CONTROLLERS

- These are corresponding helper methods defined int the **`Controller`** class to easily produce the result types, e.g.:

```
return View(...); // the result will be ViewResult
```

- A view result is usually specified with a viewmodel, which is a transformation and/or reduction of the model to the view, e.g.:

```
Object viewModel = ... // create the view model
return View("Index", viewModel);
// specify the name of the view and the viewmodel
```

  - The viewmodel can be of any type, even primitive, and can be completely independent of the original model.

EXAMPLE

Task: Create a travel agency website where you can browse apartments.

- The main page (**`Index`**) lists the basic data of the buildings, which can be filtered by the cities.
- The details page (**`Details`**) lists the apartments of a building.
- The webpage is controlled by a controller (**`HomeController`**), which defines three actions: list all buildings (**`Index`**), list the buildings of a city (**`List`**) and retrieve the details of a building (**`Details`**).
- To list the cities, we use the **`ViewBag`** property
- Data is stored in a database (**`TravelAgencyContext`**)

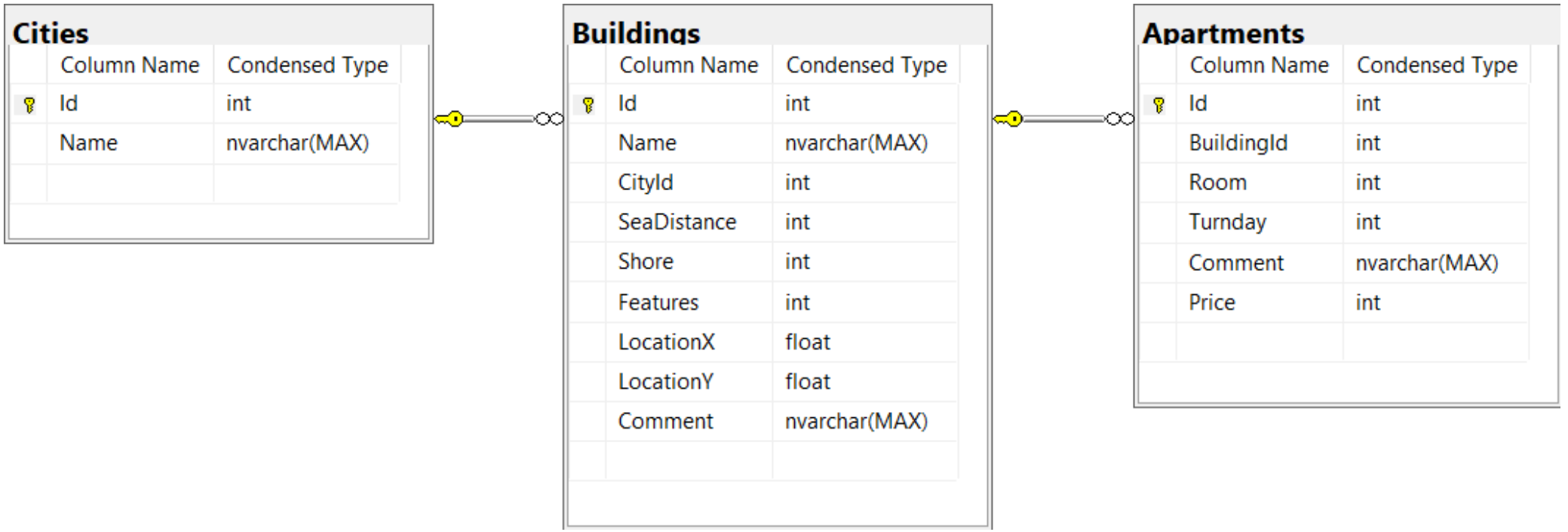
EXAMPLE

Design (*entity model*):

- we use the *Entity Framework Core* for object-relational, entity-based database management;
  - DB management will be discussed later
- three entity types will be used (**`City`**, **`Building`**, **`Apartment`**);
- the database is created in a *code-first* manner based on the entity model (if it does not exist);
- the primary keys are generated automatically by the database on persistence.

EXAMPLE

Design (*database*):



EXAMPLE

Design (*application*):

